

EXPERIMENT 4

Aim: To develop a blockchain-powered web application using Solidity programming language on Remix IDE and embedding Metamask Wallet

Theory :

Blockchain is a decentralized and distributed ledger technology that stores transactions in a secure and tamper-resistant manner. Instead of relying on a single central authority, blockchain maintains a shared record of transactions across a network of participants. One of the important applications of blockchain technology is the use of **smart contracts**, which are programs deployed on a blockchain and executed according to predefined rules.

In this experiment, a **blockchain-powered web application is developed using the Solidity programming language on Remix IDE and integrated with the MetaMask Wallet**. The experiment demonstrates how a user can create a Solidity smart contract, compile it, deploy it on a blockchain test network, and perform transactions through a MetaMask account. The given experiment specifically requires setting up MetaMask, connecting a testnet with Remix IDE, creating a simple smart contract based on the selected mini-project, compiling and deploying it, checking transaction details, and interacting with the deployed contract.

MetaMask is a cryptocurrency wallet that provides an interface for users to interact with blockchain networks and decentralized applications. It manages blockchain accounts and allows users to approve and sign transactions. In this experiment, MetaMask is used as the wallet through which transactions are performed on the selected test network.

First, MetaMask is installed as a browser extension and an account is created or imported. The wallet is then connected to a test network such as **Sepolia Testnet or RSK Testnet**. Test cryptocurrency is added to the wallet so that transactions such as smart-contract deployment and function execution can be performed. The experiment manual specifies Sepolia Testnet or RSK Testnet as the networks to be used.

A **testnet** is a blockchain network used for development and testing purposes. It provides an environment similar to a main blockchain network but uses test cryptocurrency instead of real funds. This allows developers and students to safely deploy and test smart contracts.

In this experiment, the smart contract is deployed on either the **Sepolia Testnet or RSK Testnet**. The use of a testnet makes it possible to perform transactions and observe blockchain behavior without using real cryptocurrency.

Remix IDE is a web-based development environment used to write and work with Solidity smart contracts. It provides facilities for creating Solidity source files, compiling smart contracts, deploying them to a blockchain network, and interacting with deployed contracts.

A Solidity contract is written in Remix IDE and then compiled using the Solidity compiler. Compilation verifies the program and generates the information required for deployment and interaction with the contract. After successful compilation, the contract can be deployed using the **Deploy & Run Transactions** section of Remix.

Solidity is a programming language used to develop smart contracts for Ethereum-compatible blockchain platforms. A smart contract contains variables, functions, and rules that define how information is stored and how operations are performed.

For this experiment, a **simple Solidity smart contract based on the selected mini-project** is created. The contract may contain functions to store, modify, and retrieve information depending on the requirements of the mini-project. The lab manual specifically instructs students to create a simple smart contract based on their chosen mini-project.

Connecting MetaMask with Remix IDE

MetaMask can be connected to Remix IDE so that Remix can use the blockchain account managed by MetaMask. The appropriate injected MetaMask provider is selected in Remix, and MetaMask asks the user for permission to connect.

Compilation and Deployment

After writing the Solidity smart contract, it is compiled using the Solidity Compiler in Remix IDE. If there are no compilation errors, the contract is ready for deployment.

During deployment, the user selects the required contract and uses the connected MetaMask account. When the **Deploy** operation is performed, MetaMask displays the transaction details. After the user confirms the transaction, it is submitted to the selected testnet. Once the transaction is successfully processed, the smart contract is deployed and its contract address is displayed in Remix.

Blockchain Transactions

A blockchain transaction is an operation submitted to the blockchain network. In this experiment, deploying the smart contract and executing functions that modify contract data are examples of transactions. When a transaction is initiated, MetaMask is responsible for obtaining the user's approval. After confirmation, the transaction is sent to the testnet and processed by the blockchain. Once confirmed, the changes made by the transaction become part of the blockchain's recorded state.

Checking Transaction Details

After performing transactions, their details can be checked using a blockchain explorer. The experiment instructions specifically mention checking transaction details on the **RSK Explorer**.

Transaction details can be used to verify that the transaction was successfully processed. Depending on the network and explorer, information such as transaction hash, block number, sender address, receiver or contract address, transaction status, and gas information can be viewed.

If the experiment is performed using Sepolia instead of RSK, the corresponding Sepolia blockchain explorer should be used to inspect the transaction.

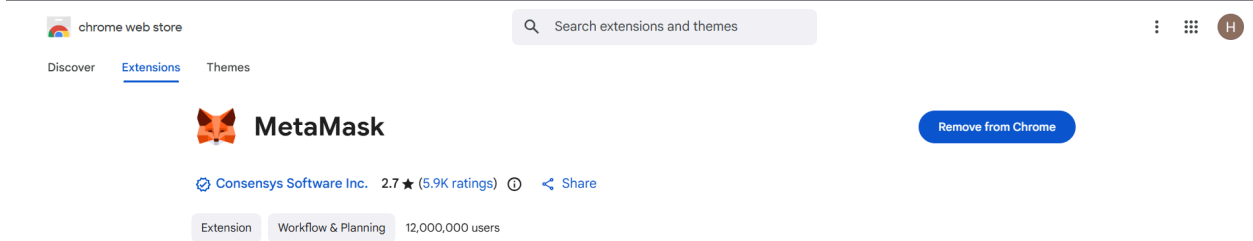
Interaction with the Smart Contract

After deployment, Remix provides an interface through which the functions of the smart contract can be accessed. Functions that modify the contract's stored information require a blockchain transaction and therefore require confirmation through MetaMask.

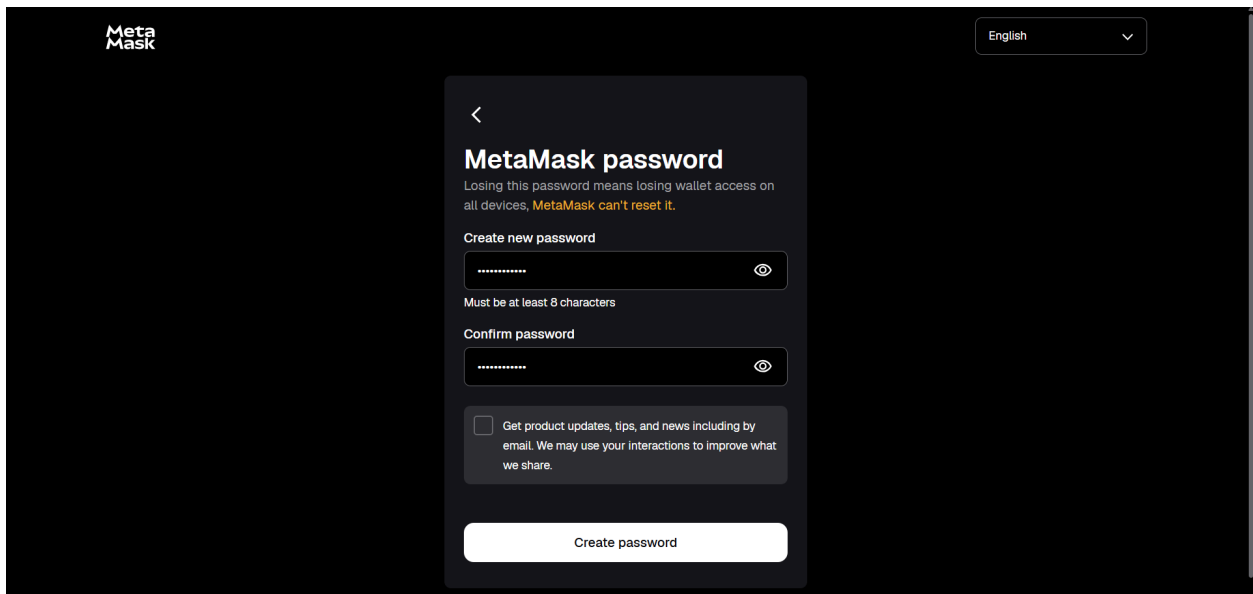
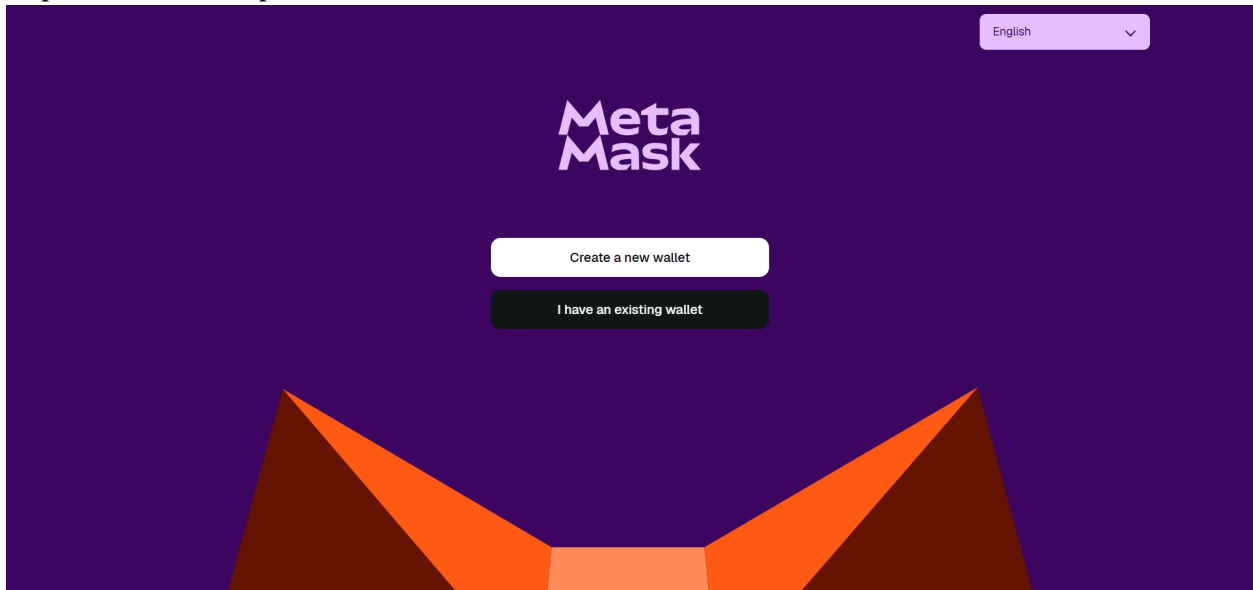
Other functions that only retrieve information from the blockchain can be called to observe the current state of the contract. By executing these functions, the user can verify that the smart contract has been deployed correctly and that the expected information has been stored or updated.

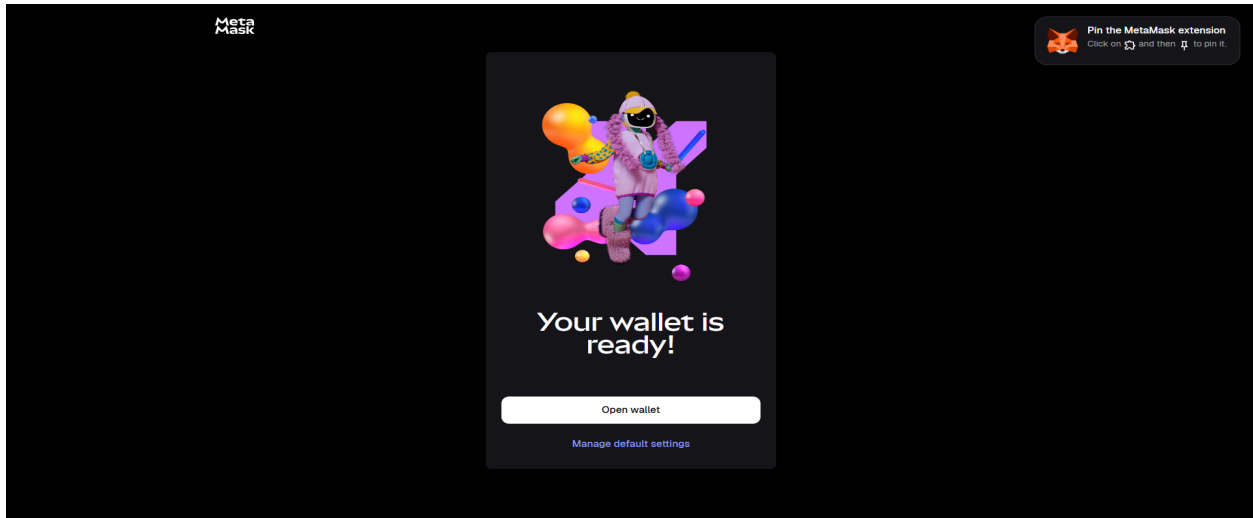
Code and Output :

Step 1: Install MetaMask

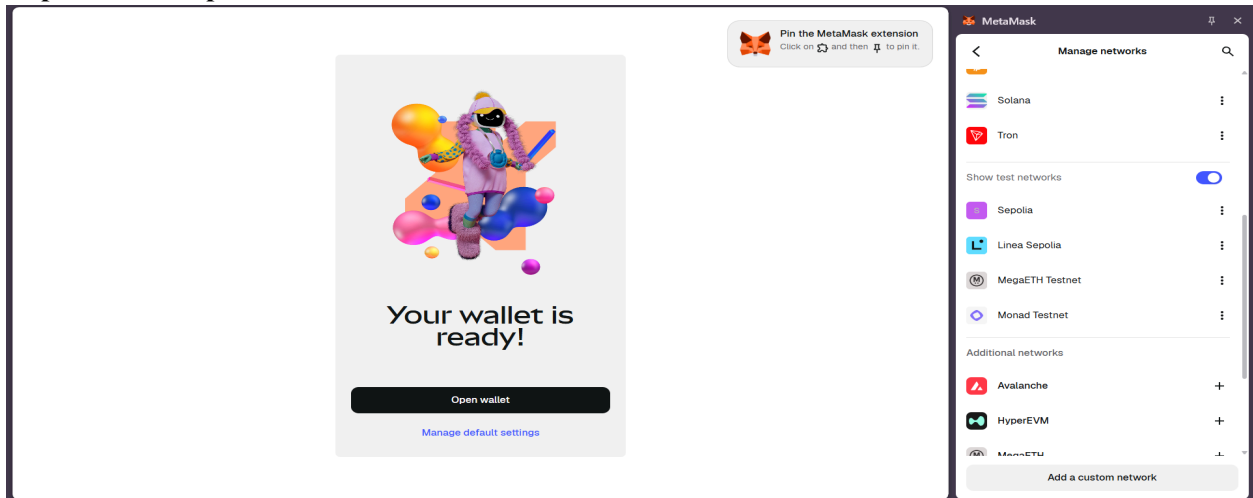


Step 2: Create or Import an Account

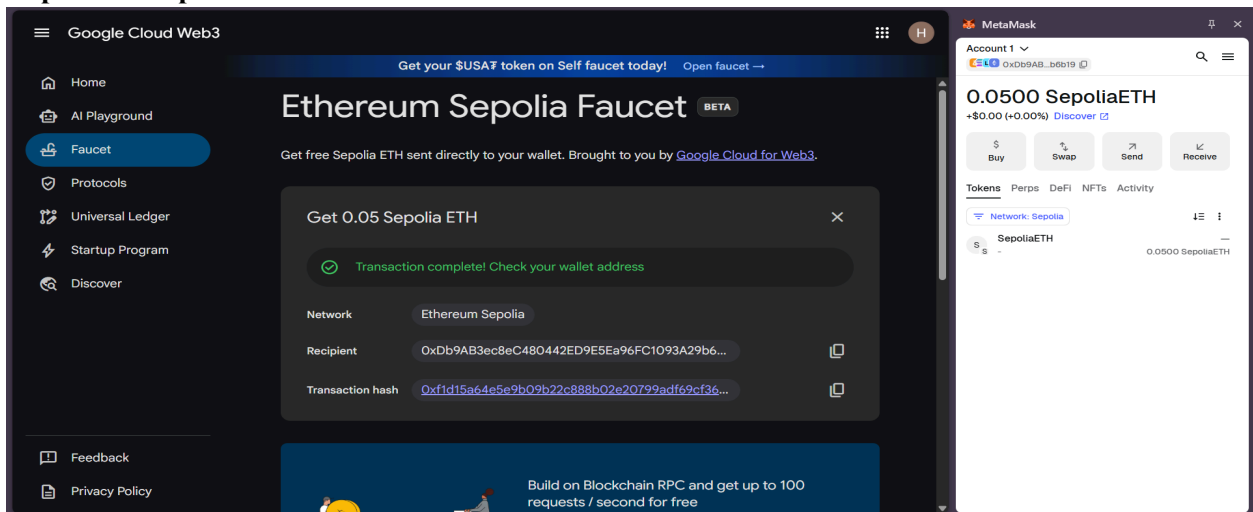




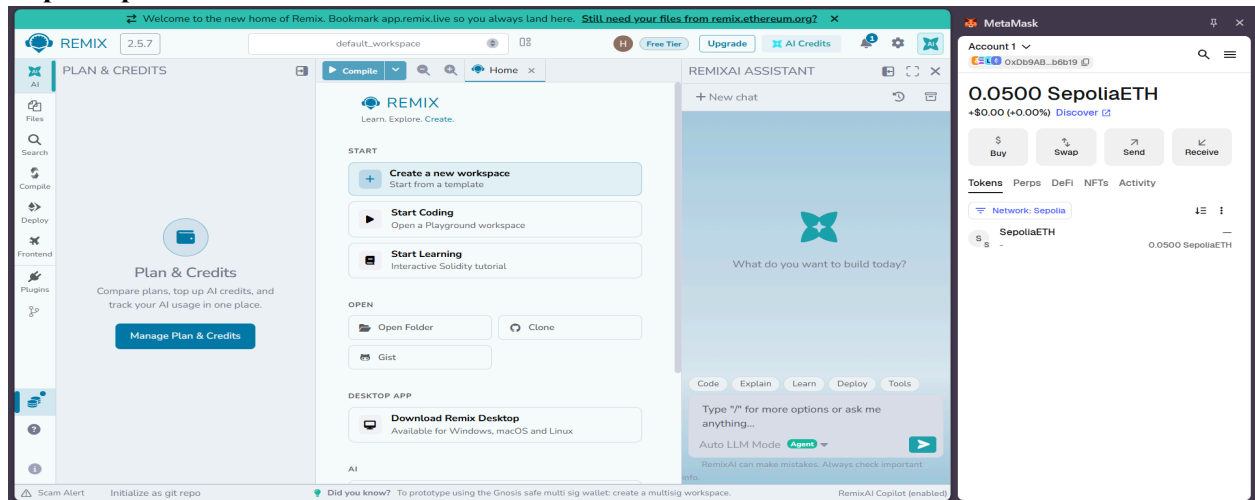
Step 3: Select Sepolia



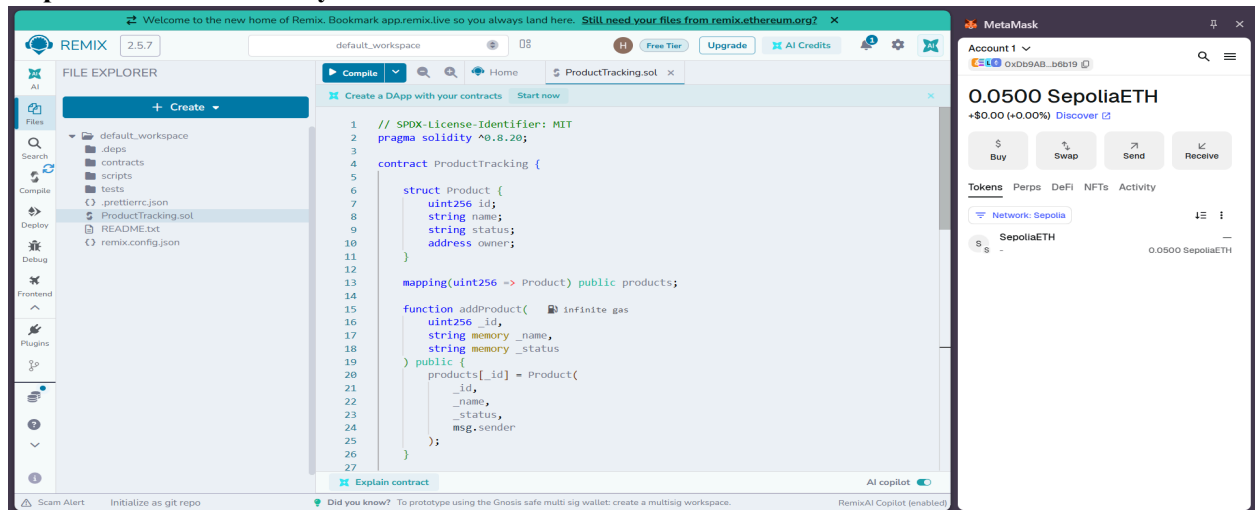
Step 4: Get Sepolia ETH



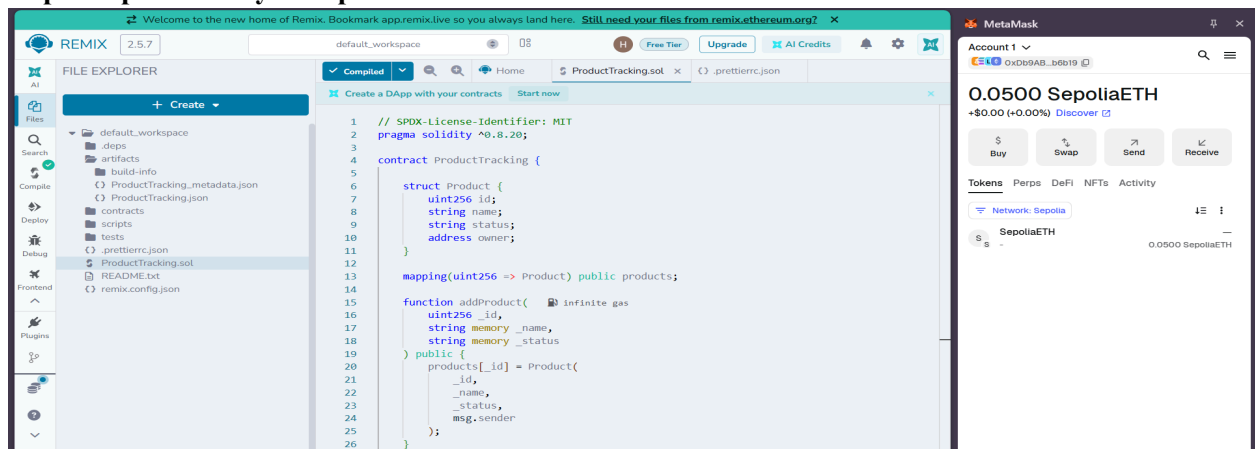
Step 5: Open Remix



Step 6: Create the Solidity file



Step 7: Open Solidity Compiler



Step 8: Open Deploy & Run Transactions

The screenshot shows the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The 'Environment' dropdown is set to 'Remix VM osaka'. The 'ProductTracking' contract is selected, and its status is 'Compiled'. The 'Value' is set to '0' and the 'Gas limit' is 'auto'. The 'Deploy' button is visible. The 'Deployed Contracts' section shows 'There is no contract to show.' The main editor displays the Solidity code for the 'ProductTracking' contract. The MetaMask wallet is connected to 'app.remix.live' and shows 'Account 1' with a balance of '0.0500 SepoliaETH'. The permissions section shows 'See addresses, balances, and activity' and 'Send requests for transactions' as granted, while 'Cannot move funds without your permission' is denied.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract ProductTracking {
5
6     struct Product {
7         uint256 id;
8         string name;
9         string status;
10        address owner;
11    }
12
13    mapping(uint256 => Product) public products;
14
15    function addProduct(
16        uint256 _id,
17        string memory _name,
18        string memory _status
19    ) public {
20        products[_id] = Product(
21            _id,
22            _name,
23            _status,
24            msg.sender
25        );
26    }
27 }
```

Step 9: Connect MetaMask

The screenshot shows the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The 'Environment' dropdown is set to 'Sepolia Testnet - MetaMask'. The 'ProductTracking' contract is selected, and its status is 'Compiled'. The 'Value' is set to '0' and the 'Gas limit' is 'auto'. The 'Deploy' button is visible. The 'Deployed Contracts' section shows 'There is no contract to show.' The main editor displays the Solidity code for the 'ProductTracking' contract. The MetaMask wallet is connected to 'app.remix.live' and shows 'Account 1' with a balance of '0.0500 SepoliaETH'. The permissions section shows 'See addresses, balances, and activity' and 'Send requests for transactions' as granted, while 'Cannot move funds without your permission' is denied.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract ProductTracking {
5
6     struct Product {
7         uint256 id;
8         string name;
9         string status;
10        address owner;
11    }
12
13    mapping(uint256 => Product) public products;
14
15    function addProduct(
16        uint256 _id,
17        string memory _name,
18        string memory _status
19    ) public {
20        products[_id] = Product(
21            _id,
22            _name,
23            _status,
24            msg.sender
25        );
26    }
27 }
```

Step 10: Deploy Smart Contracts

The screenshot shows the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The 'Environment' section shows 'Sepolia Testnet - MetaMask' and 'Account 16' with a balance of 0.050 ETH. The 'Deploy' section shows the 'ProductTracking' contract, which has been compiled. The 'Value' is set to 0 and the 'Gas limit' is set to 'auto'. The 'Deploy' button is visible. The main editor displays the Solidity code for the 'ProductTracking' contract. The MetaMask wallet on the right shows the 'Deploy a contract' dialog, indicating that the site wants to deploy a contract. The 'Network' is set to 'Sepolia' and the 'Request from' is 'app.remix.live'. The 'Network fee' is 0.0018 SepoliaETH and the 'Speed' is 'Market ~12 sec'. The 'Confirm' button is highlighted.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract ProductTracking {
5
6     struct Product {
7         uint256 id;
8         string name;
9         string status;
10        address owner;
11    }
12
13    mapping(uint256 => Product) public products;
14
15    function addProduct(
16        uint256 _id,
17        string memory _name,
18        string memory _status
19    ) public {
20        products[_id] = Product(
21            _id,
22            _name,
23            _status,
24            msg.sender
25        );
26    }
27 }
```

Step 11: Verify Deployment

The screenshot shows the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The 'Environment' section shows 'Sepolia Testnet - MetaMask' and 'Account 16' with a balance of 0.048 ETH. The 'Deployed Contracts' section shows the 'ProductTracking' contract, which has been deployed. The 'Functions' section shows a list of functions, including 'addProduct'. The 'Verify Contract on Explorers' button is visible. The main editor displays the Solidity code for the 'ProductTracking' contract. The MetaMask wallet on the right shows the '0.0482 SepoliaETH' balance. The 'Activity' tab is selected, showing a 'Contract Interaction' with the contract.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract ProductTracking {
5
6     struct Product {
7         uint256 id;
8         string name;
9         string status;
10        address owner;
11    }
12
13    mapping(uint256 => Product) public products;
14
15    function addProduct(
16        uint256 _id,
17        string memory _name,
18        string memory _status
19    ) public {
20        products[_id] = Product(
21            _id,
22            _name,
23            _status,
24            msg.sender
25        );
26    }
27 }
```

Step 12: Execute addProduct() and Approve the Transaction

The screenshot shows the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The 'Environment' section shows 'Sepolia Testnet - MetaMask' and 'Account 16' with a balance of 0.048 ETH. The 'addProduct' function is being executed with parameters: uint256, string, string. The 'Transact' button is visible. The main editor shows Solidity code for a 'ProductTracking' contract. The MetaMask wallet shows a 'Transaction request' for 0.0007 SepoliaETH, with a 'Confirm' button.

Remix IDE - Environment:

- Browser Extension: Sepolia Testnet - MetaMask
- Account 16: 0.048 ETH

Remix IDE - addProduct Function:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract ProductTracking {
5
6     struct Product {
7         uint256 id;
8         string name;
9         string status;
10        address owner;
11    }
12
13    mapping(uint256 => Product) public products;
14
15    function addProduct(
16        uint256 _id,
17        string memory _name,
18        string memory _status
19    ) public {
20        products[_id] = Product(
```

MetaMask - Transaction request:

- Balance changes: No changes
- Added protection: Optional (checked)
- Network: Sepolia
- Request from: app.remix.live
- Interacting with: 0xA3ad0...355A7
- Network fee: 0.0007 SepoliaETH
- Speed: Market -12 sec

The screenshot shows the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The 'Environment' section shows 'Sepolia Testnet - MetaMask' and 'Account 16' with a balance of 0.047 ETH. The 'Transact' button is visible. The main editor shows Solidity code for a 'ProductTracking' contract. The MetaMask wallet shows a 'Transaction request' for 0.0477 SepoliaETH, with a 'Confirm' button. Below the transaction request, there is a 'Transactions recorder 2' section showing a list of transactions, including 'ProductTracking' and 'addProduct'.

Remix IDE - Environment:

- Browser Extension: Sepolia Testnet - MetaMask
- Account 16: 0.047 ETH

Remix IDE - Transactions recorder 2:

Contract call	Transaction list	Oldest
ProductTracking	tx: 0x6ee...beab6	7min ago
addProduct	tx: 0x2a5...ca03a	1min ago

MetaMask - Transaction request:

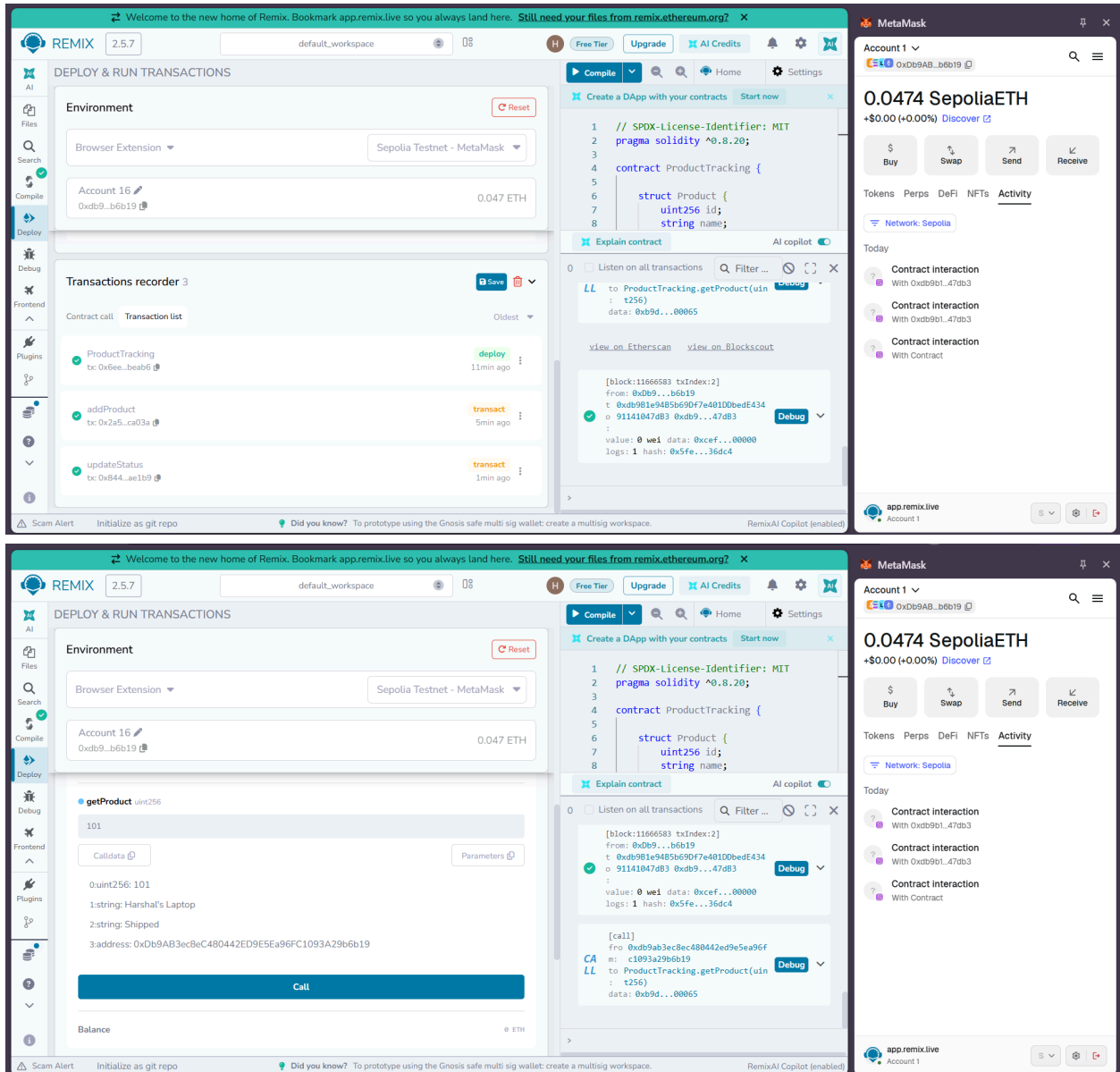
- Balance changes: +0.000 (+0.00%)
- Network: Sepolia
- Today: Contract interaction (With 0xDb9b1...47db3)

Step 13: Use getProduct()

The screenshot displays the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'Environment' panel on the left shows the 'getProduct' function being called with the parameter 'uint256: 101'. The 'Call' button is visible. The 'Compiler' panel in the center shows the Solidity code for the 'ProductTracking' contract, which includes a 'getProduct' function. The 'Debug' panel on the right shows the transaction details, including the block number (11666567), transaction index (17), and the function call 'ProductTracking.getProduct(uint256: 101)'. The MetaMask wallet on the right shows the account balance of 0.0477 SepoliaETH and a list of recent transactions, including the one that was just executed.

Step 14: Use updateStatus()

The screenshot displays the Remix IDE interface on the left and the MetaMask wallet on the right. In the Remix IDE, the 'Environment' panel on the left shows the 'updateStatus' function being called with the parameter 'uint256, string: 0'. The 'Transact' button is visible. The 'Compiler' panel in the center shows the Solidity code for the 'ProductTracking' contract, which includes an 'updateStatus' function. The 'Debug' panel on the right shows the transaction details, including the block number (11666567), transaction index (17), and the function call 'ProductTracking.updateStatus(uint256: 0, string: 0)'. The MetaMask wallet on the right shows a 'Transaction request' dialog with a balance change of 0.0004 SepoliaETH and a network fee of 0.0004 SepoliaETH. The 'Confirm' button is visible.



Conclusion :

Thus, a blockchain-based application was successfully developed using Solidity and Remix IDE. MetaMask was connected to the selected testnet, the smart contract was compiled and deployed, and transactions were successfully performed and verified. The experiment provided practical understanding of MetaMask, testnets, smart-contract deployment, blockchain transactions, and interaction with Solidity smart contracts.